॥वसुधैव कुटुम्बकम्॥

# SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)

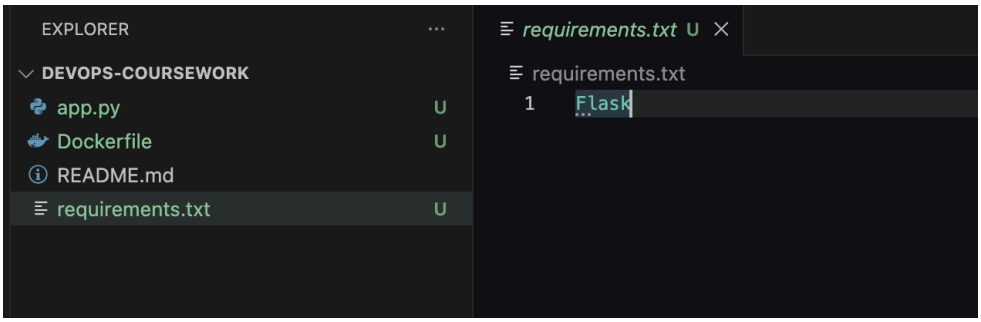
**Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC**

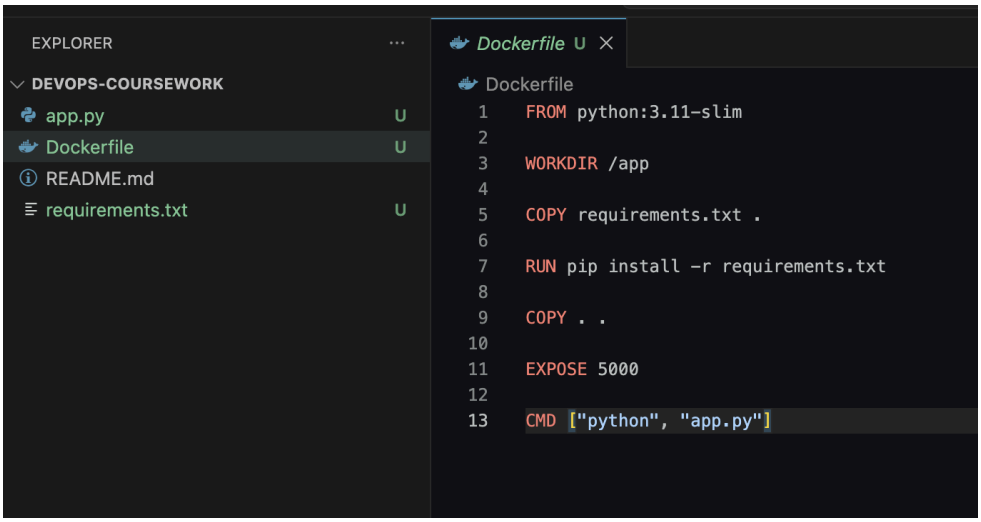
**Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)**

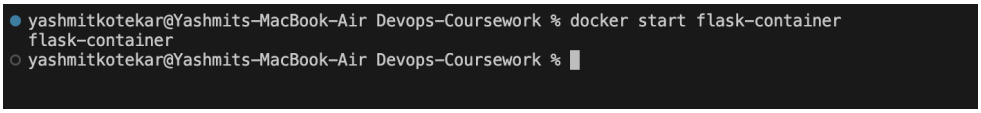
## Assignment No. 08

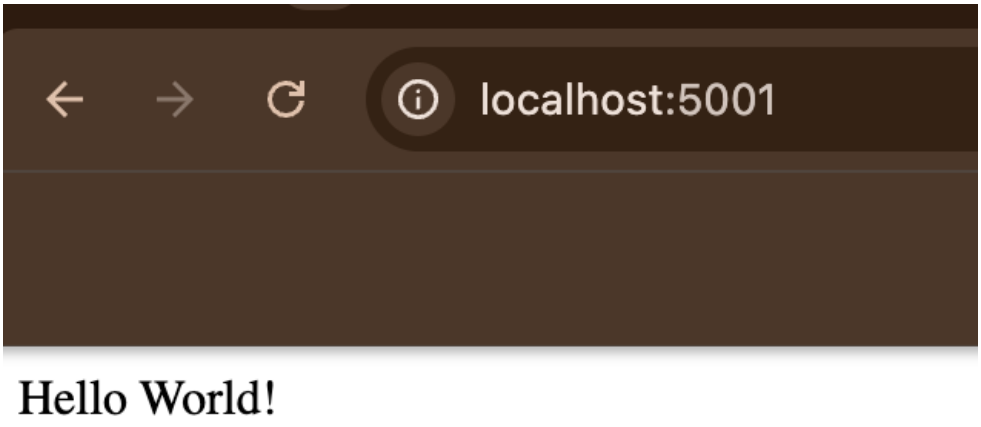
| | |
|---|---|
| **Subject:** | DevOps Lab |
| **TCODE:** | TE7950 |
| **Name of Student** | **Yashmit Kotekar** |
| **PRN No.** | **23070122236** |
| **Branch** | CSE, Batch (2023-27), L4 |
| **Academic Year & Semester** | 25 – 26, Sem VII |
| **Date of Performance** | 23rd July, 2026 |
| **Title of Assignment:** | Dockerfile & Image Build |
| **Theory:** | Docker is a containerization platform that enables applications and their dependencies to be packaged into lightweight, portable containers. A Dockerfile contains the instructions required to build a Docker image, while a Docker image serves as a template for creating containers. A Docker container is a running instance of an image that provides an isolated environment for executing applications consistently across different systems. In this experiment, a simple Flask web application was containerized by creating the required application files, building a Docker image, running the container, and verifying the application through a web browser.<br><br>**Commands Used**<br>• mkdir my-flask-app – Created a new directory for the Flask application.<br>• cd my-flask-app – Navigated to the project directory.<br>• docker build -t my-flask-app . – Built a Docker image named my-flask-app from the Dockerfile.<br>• docker build --no-cache -t my-flask-app . – Rebuilt the Docker image without using cached layers after updating the dependencies.<br>• docker run -p 5000:5000 my-flask-app – Started the Docker container and mapped port 5000 of the container to port 5000 on the host machine for browser access. |

| | |
|---|---|
| **Implementation:** | **Step 1: Create the Flask Application**<br><br>Created Flask Application — Created the my-flask-app project directory and added the required application files. Implemented the app.py file containing a simple Flask web application that displays the message "Hello, Docker!" when accessed.<br><br>**Step 2: Create the Requirements File**<br><br>Created Requirements File — Created the requirements.txt file specifying the Flask dependency required to install the necessary packages for the Flask application.<br><br>**Step 3 – Create the Dockerfile**<br><br>Created Dockerfile — Created the Dockerfile specifying the Python base image, working directory, application dependencies, exposed port, and startup command required to containerize the Flask application.<br><br>**Step 4 – Build the Docker Image**<br><br>Built Docker Image — Successfully built the Docker image named my-flask-app using the Dockerfile. Docker completed all build stages and created the container image successfully.<br><br>**Step 5 – Run the Docker Container**<br><br>Executed Docker Container — Successfully started the Docker container and launched the Flask application. The application is running on port 5000 and is accessible through the host machine.<br><br>**Step 6: Verify the Flask Application**<br><br>Verified Dockerized Flask Application — Opened the Flask application in a web browser using http://localhost:5000 and successfully verified that the Docker container served the "Hello, Docker!" webpage.<br><br>app.py<br>`from flask import Flask`<br><br>`app = Flask(__name__)`<br><br>`@app.route("/")`<br>`def hello():`<br>`    return "Hello World!"`<br><br>`if __name__ == "__main__":`<br>`    app.run(host="0.0.0.0", port=5000)` |




Subject Faculty Name: Prof. Arijit Dutta